Работа выполняется в удобной Вам среде на языке программирования Python.

В случае работы с блокнотом в среде Google (https://colab.google/) или jupiter notebook каждая из исполняемых ячеек должна предваряться заголовком, в котором размещено описание задания. Вывод результата осуществляется либо средствами среды, либо вызовом функции print()

Если работаете в PyCharm то будем считать что уже опытные пользователи и можете сделать что то чуть более сложное. Каждое задание должно быть оформлено в виде отдельной функции. Датасет должен открываться только один раз при запуске программы. Выбор задания, результат которого будет выведен в консоль осуществляется в той же консоли путем задания числового или строкового значения, соответствующего номеру задания. Например, 1 или «NumPy_1», 9 или «Pandas_4».

Выполненные задания (блокноты или папки с проектами PyCharm без датасета Titanic) присылаете из-под своего аккаунта в системе moodle. Оценка не ставится - учитывается факт выполнения задания - зачет/не зачет. Основная цель - получение Вами навыков.

Python. Основные навыки

1. Вычисление суммы ряда: Напишите код, который вычисляет сумму последовательности чисел Фибоначчи до определенного предела, указанного пользователем.
   Числа Фибоначчи (строка Фибоначчи) — числовая последовательность, первые два числа которой являются 0 и 1, а каждое последующее за ними число является суммой двух предыдущих. Представляет собой частный пример линейной рекуррентной последовательности (рекурсии).
2. Генерация простых чисел: Создайте функцию, которая генерирует список простых чисел в диапазоне от 2 до заданного числа.
   Простое число — это натуральное число, большее единицы, имеющее ровно два натуральных делителя: 1 и само себя.
3. Работа с файлами: Прочитайте содержимое файла data.txt и выведите его на экран. Затем сохраните преобразованный текст обратно в файл.
4. Проверка корректности ввода: Напишите программу, которая проверяет, является ли переданное число положительным целым числом. Если условие выполняется, программа должна вывести его квадратный корень.
5. Поиск в списке: Дан список строк. Напишите функцию, которая ищет в нем все строки, содержащие определенное слово, и возвращает их количество.
   Список строк сформируйте из загружаемого файла. Файл и его наполнение выбираете на свое усмотрение.
6. Манипуляции со строками: Преобразуйте строку так, чтобы каждое слово начиналось с заглавной буквы, а все остальные символы были строчными.
   Список строк сформируйте из загружаемого файла. Файл и его наполнение выбираете на свое усмотрение.

7. Словарь частот: Напишите программу, которая считывает текстовый файл и строит словарь, где ключами являются уникальные слова, а значениями — частота их появления в тексте.
8. Обработка списка: Дан список чисел. Удалите из него все дублирующиеся элементы и отсортируйте оставшиеся в порядке возрастания.
   Список формируете на свое усмотрение для демонстрации работы кода.
9. Максимальное и минимальное значение в списке: Напишите функцию, которая принимает список чисел и возвращает максимальное и минимальное значения из этого списка.
   Список формируете на свое усмотрение для демонстрации работы кода.
10. Подсчет вхождений символа в строке: Напишите функцию, которая считает количество вхождений определенного символа в заданную строку.
    Строки возьмите из файла, наполнение которого остается за Вами. Из него сформируйте массив строк. В каждой строке определите вхождение искомого символа. Символ для поиска также выберете сами. Результат работы должен быть в формате: Строка 1 - ... вхождений \n Строка 2 - ... вхождений и т.д.
11. Удаление пробелов из строки: Напишите функцию, которая удаляет все лишние пробелы из строки (оставляя только один пробел между словами).
12. Разделение строки на части: Напишите функцию, которая разбивает строку на отдельные слова и возвращает их в виде списка.
13. Перевод строки в обратный порядок: Напишите функцию, которая переворачивает строку (например, "привет" становится "тевирп"). Добавьте функцию, определяющую является ли строка палиндромом.
    Палиндром - Слово или фраза, которые одинаково читаются слева направо и справа налево.
14. Операции над множествами: Определите два множества и выполните операции объединения, пересечения и разности этих множеств.
15. Факторизация числа: Напишите функцию, которая находит все делители целого числа.
    Делитель целого числа n — это целое число m, для которого n/m является целым числом (которое также является делителем n). Например, 3 является делителем числа 21, поскольку 21/3 = 7 (и 7 также является делителем числа 21).
16. Использование функций: Напишите функцию is_prime(n), которая определяет, является ли число простым. Используйте эту функцию для проверки всех чисел в диапазоне от 2 до 100.
17. Циклические конструкции: Создайте программу, которая выводит таблицу умножения (например, от 1 до 10) с использованием вложенных циклов.
18. Обработка исключений: Напишите программу, которая запрашивает у пользователя ввод целых чисел и пытается разделить одно число на другое. Если возникает ошибка деления на ноль, программа должна обработать исключение и предложить ввести другое число.
19. Модули и импортирование: Используя модуль random, напишите программу, которая генерирует случайное число в заданном диапазоне и предлагает

пользователю угадать это число. Программа должна подсказывать, больше или меньше введенное число относительно загаданного.

20. Изменение размера изображения: Напишите скрипт, который открывает изображение (используется библиотека PIL), уменьшает его размер вдвое и сохраняет новое изображение.
21. Переворот изображения: Откройте изображение и переверните его вертикально или горизонтально. Сохраните результат в новый файл.
22. Сумма четных чисел: Напишите программу, которая вычисляет сумму всех четных чисел в диапазоне от 1 до N (где N вводится пользователем).
23. Вычисление степеней двойки: Используя цикл, вычислите степени двойки от 1 до 10 (2^1, 2^2, ..., 2^10) и выведите результаты.
24. Поиск простых чисел в диапазоне: Найдите все простые числа в диапазоне от M до N (вводятся пользователем) с использованием цикла.
25. Фибоначчи с циклом: Выведите первые N чисел Фибоначчи, используя цикл (где N задается пользователем).
26. Шаги по таблице ASCII: Выведите все символы таблицы ASCII от 'A' до 'Z', используя цикл.
27. Частота символов в строке: Напишите программу, которая подсчитывает частоту каждого символа в строке и сохраняет результат в словарь.
28. Объединение словарей: Даны два словаря. Объедините их таким образом, чтобы ключи оставались уникальными, а значения суммировались.
29. Максимальная длина ключа: Для данного словаря найдите ключ с максимальной длиной.
30. Подсчет уникальных значений: Подсчитайте количество уникальных значений в словаре.
31. Удаление элементов с определенным значением: Удалите из словаря все пары ключ-значение, где значение равно заданному числу.
32. Создание множества из списка: Преобразуйте список в множество и выведите его.
33. Операции над множествами: Выполните операции объединения, пересечения и симметричной разницы двух множеств.
34. Проверка на подмножество: Проверьте, является ли одно множество подмножеством другого.
35. Симметричная разница: Найдите симметричную разницу двух множеств.
36. Минимум и максимум: Найдите минимальный и максимальный элемент во множестве.

NumPy

1. Создайте одномерный массив из чисел от 0 до 9, преобразуйте его в двумерный массив размером 3x3, затем найдите сумму всех элементов, умножьте полученную сумму на 2 и выведите результат.

2. Создайте двумерный массив размером 4x4, заполненный нулями. Добавьте единичные значения на главную диагональ, затем удалите первую строку и последний столбец, после чего найдите сумму оставшихся элементов.
3. Создайте массив случайных целых чисел размером 10x10, отсортируйте его строки по возрастанию, затем найдите минимальные значения в каждой строке, замените их на максимальные значения соответствующей строки и подсчитайте количество изменений.
4. Найдите минимальный элемент в массиве, замените его на максимальное значение, затем умножьте весь массив на 2, после чего отсортируйте массив по убыванию.
5. Объедините два массива одинакового размера по горизонтали, затем объедините результат по вертикали с третьим массивом, отсортируйте итоговый массив по строкам и найдите среднее значение по столбцам.
6. Вычислите среднее значение элементов массива, добавьте это значение ко всем элементам массива, найдите новые средние значения по строкам, затем отсортируйте строки по новым средним значениям.
7. Найдите индексы всех отрицательных элементов в массиве, замените эти элементы на их абсолютные значения, вычислите сумму новых положительных элементов и сравните ее с суммой исходных элементов.
8. Преобразуйте одномерный массив в двумерный с заданной формой, затем переверните его по вертикали и по горизонтали, вернитесь к исходному виду и сравните с исходным массивом.
9. Извлеките каждый третий элемент из одномерного массива, создайте новый массив, состоящий из этих элементов, сравните его с исходным массивом и найдите разницу между этими массивами.
10. Проверить, содержит ли массив хотя бы одно положительное число. Если да, удвоить положительные числа, иначе добавить единицу ко всем элементам, затем найти количество нулевых элементов.
11. Отсортируйте элементы массива по возрастанию, затем разбейте массив на четыре равные части, найдите минимальное значение в каждой части и замените минимальные значения на максимальные значения соответствующей части.
12. Умножьте каждый элемент массива на заданное число, затем округлите полученные значения до ближайшего целого, найдите уникальные элементы и отсортируйте их по возрастанию.
13. Переставьте строки и столбцы в массиве местами, затем удалите вторую строку и третий столбец, снова переставьте строки и столбцы и сравните полученный массив с исходным.
14. Найдите количество неповторяющихся элементов в массиве, отсортируйте массив, найдите среднее значение неповторяющихся элементов и замените их на это среднее значение.
15. Разделите массив на две части по среднему значению, соедините обе части обратно, отсортируйте полученный массив и найдите медиану.

16. Удалите все повторяющиеся значения из массива, преобразуйте оставшийся массив в вектор-столбец, вычислите его длину и сравните с длиной исходного массива.
17. Найдите максимальный элемент в каждой строке двумерного массива, замените найденные максимумы на минимальные значения соответствующих строк, затем найдите разность между новыми и старыми максимальными значениями.
18. Найдите произведение элементов массива вдоль главной диагонали, добавьте это произведение к каждому элементу массива, затем отсортируйте массив по строкам.
19. Создайте треугольную матрицу заданного размера, поверните её на 180 градусов, сложите с исходной матрицей и найдите максимальную сумму по строкам.
20. Сравните два массива и верните индексы различающихся элементов, замените эти элементы на их среднее арифметическое, затем найдите разницу между исходными и новыми массивами.
21. Примените функцию к каждому элементу массива, выберите только четные элементы, найдите их сумму и сравните ее с суммой нечетных элементов.
22. Поверните массив на 90 градусов против часовой стрелки, извлеките элементы, лежащие на границе полученного массива, и замените их на нули, затем поверните массив обратно.
23. Проверьте, является ли массив симметричным относительно главной диагонали, транспонируйте его, проверьте симметрию ещё раз и сравните результаты проверок.
24. Найдите сумму каждого столбца в двумерном массиве, постройте новый массив, где каждый элемент равен сумме соответствующего столбца, затем сравните новый массив с исходным.
25. Найдите разницу между максимальным и минимальным элементами массива, умножьте эту разницу на все элементы массива, затем отсортируйте массив по возрастанию.
26. Убедитесь, что массив состоит только из целых чисел, возведите все элементы в куб, найдите сумму получившихся значений и сравните ее с суммой исходных элементов.
27. Извлеките квадратные корни из всех элементов массива, округлите результаты до двух знаков после запятой, отсортируйте массив по убыванию и сравните с исходным массивом.
28. Найдите индекс первого положительного элемента в массиве, замените этот элемент на его квадратный корень, отсортируйте массив и сравните его с исходным массивом.
29. Преобразуйте массив строк в массив чисел, отсортируйте его, найдите медиану и сравните её с медианой исходного массива.

Pandas

I. Работа с набором данных Titanic (описание находится здесь):
    1. Загрузка и просмотр данных:
        a. Загрузите датасет Titanic.csv с помощью функции pd.read_csv().

   b. Выведите первые пять строк с помощью метода .head().
   c. Определите количество записей и столбцов с помощью методов .shape и .info().
   d. Найдите уникальные значения в столбце Sex с помощью метода .unique().
   e. Посмотрите описание данных с помощью метода describe().
2. Фильтрация данных
   a. Отфильтруйте данные, оставив только тех пассажиров, кто выжил (Survived == 1).
   b. Из полученных данных выберите только столбцы Name, Age и Sex.
   c. Сгруппируйте выживших пассажиров по полу и посчитайте среднее значение возраста.
   d. Фильтруйте данные по условию Age > 50.
   e. Сохраните результат фильтрации в отдельный DataFrame.
3. Работа с пропущенными значениями
   a. Найдите количество пропущенных значений в каждом столбце с помощью метода .isnull().sum().
   b. Заполните пропущенные значения в столбце Age медианой возраста среди всех пассажиров.
   c. Удалите строки с пропущенными значениями в столбцах Cabin и Embarked.
   d. Проверьте, остались ли пропущенные значения после заполнения.
   e. Создайте новую копию DataFrame без пропущенных значений.
4. Создание новых столбцов
   a. Создайте новый столбец Title, извлекая титул пассажира из имени (например, Mr., Mrs.).
   b. Добавьте новый столбец FamilySize, который будет равен сумме количества братьев/сестер и родителей/детей.
   c. Посчитайте долю выживших среди мужчин и женщин отдельно.
   d. Присвойте новые имена столбцам с помощью метода .rename().
   e. Экспортируйте полученный DataFrame в CSV-файл.
5. Агрегация данных
   a. Посчитайте общее количество пассажиров, погибших и выживших.
   b. Рассчитайте средний возраст пассажиров в зависимости от класса билета (Pclass).
   c. Постройте гистограмму распределения возрастов пассажиров.
   d. Нарисуйте круговую диаграмму доли выживших по классам билетов.
   e. Сохраните графики в формате PNG.
6. Сортировка данных
   a. Отсортируйте данные по возрасту пассажиров в порядке убывания.

    b. Выберите пассажиров, которым больше 60 лет, и отсортируйте их по стоимости билета (Fare) в порядке возрастания.
    c. Подсчитайте количество пассажиров старше 18 лет в каждом классе билетов.
    d. Экспорт данных в Excel.
    e. Создание сводной таблицы по количеству выживших и погибших.
7. Применение функций к данным
    a. Примените функцию округления до ближайшего целого числа к столбцу Age.
    b. Преобразуйте числовые значения пола в категориальные (male и female).
    c. Проверьте наличие дубликатов в данных и удалите их, если такие есть.
    d. Переведите все значения в столбце Name в нижний регистр.
    e. Соедините столбцы First Name и Last Name в один столбец Full Name.
8. Обработка строковых данных
    a. Измените формат имен пассажиров, сделав их заглавными буквами.
    b. Уберите лишние пробелы в названиях портов посадки (Embarked).
    c. Замените все пустые значения в столбце Cabin на строку "Unknown".
    d. Найдите самые длинные и короткие имена пассажиров.
    e. Разделите имена на три столбца: Title, First Name, Last Name.
9. Группировка данных
    a. Сгруппируйте данные по полу и классу билета, подсчитывая количество пассажиров в каждой группе.
    b. Вычислите процент выживших пассажиров в каждой возрастной категории (0–10, 11–20, 21–30 и т.д.).
    c. Постройте график, показывающий зависимость процента выживших от класса билета.
    d. Создайте сводную таблицу по количеству выживших и погибших по классам билетов.
    e. Сделайте вывод о связи класса билета и шансов на выживание.
10. Объединение и слияние данных
    a. Объедините два DataFrame, содержащие информацию о пассажирах, по идентификатору (PassengerId).
    b. Выполните левое объединение двух DataFrame по полю Name.
    c. После объединения создайте новый столбец, содержащий разницу между возрастом пассажиров в объединенном наборе данных.
    d. Проверьте уникальность значений в новом DataFrame.
    e. Сохраните объединенный DataFrame в файл формата JSON.
11. Отображение данных с помощью графика
    a. Постройте график распределения пассажиров по возрасту.

b. Добавьте линии тренда для каждого класса билета.
c. Покажите распределение выживших и погибших на одном графике.
d. Сохраните график в формате PDF.
e. Объясните полученные результаты.

12. Анализ зависимостей с помощью графиков
    a. Постройте диаграмму рассеяния, отображающую зависимость стоимости билета от возраста пассажира.
    b. Добавьте цвета для обозначения классов билетов.
    c. Нанесите линии регрессии для каждого класса.
    d. Сохраните график в формате SVG.
    e. Опишите наблюдаемые закономерности.
13. Анализ влияния различных факторов на выживаемость
    a. Разделите пассажиров на группы по полу и классу билета.
    b. Для каждой группы рассчитайте процент выживших.
    c. Постройте столбчатую диаграмму для сравнения процентов выживших по группам.
    d. Сравните результаты и сделайте выводы о влиянии пола и класса билета на вероятность выживания.
14. Разведывательный анализ данных (EDA)
    a. Проведите разведывательный анализ данных, исследуя распределение значений в ключевых столбцах (например, Age, Fare, SibSp, Parch).
    b. Постройте гистограммы для количественных признаков и барчарты для категориальных признаков.
    c. Найдите аномалии и выбросы в данных.
    d. Сделайте выводы о качестве и структуре данных.
15. Базовая предобработка данных
    a. Загрузите датасет Titanic.csv и проверьте наличие пропущенных значений с помощью метода .isnull().sum().
    b. Заполните пропущенные значения в столбце Age медианой возраста среди всех пассажиров.
    c. Преобразуйте категориальные переменные в числовые с помощью метода .map() или .replace(). Например, замените значения в столбце Sex на 0 (для мужчин) и 1 (для женщин).
    d. Удалите столбцы, которые содержат много пропущенных значений или не несут полезной информации для анализа (например, Cabin).
    e. Сохраните полученные данные в новый CSV-файл.

II. Работа с набором Video Game Sales (описание находится здесь):

1. Анализ продаж по платформам
    a. Загрузите датасет с помощью pd.read_csv().

   b. Отфильтруйте данные, исключив строки с отсутствующими значениями в столбце Global_Sales.
   c. Сгруппируйте данные по столбцу Platform, суммируя продажи по каждой платформе.
   d. Отсортируйте платформы по объему продаж и выведите 3 лучшие и 3 худшие платформы.
   e. Постройте горизонтальную бар-диаграмму, отображающую продажи по платформам.
   f. Сохраните диаграмму в формате PNG.
   g. Выведите суммарные продажи для каждой платформы в консоль.
2. Популярные жанры
   a. Загрузите датасет.
   b. Сгруппируйте данные по жанрам (Genre), суммируя продажи по каждому жанру.
   c. Отсортируйте жанры по суммарным продажам и выведите топ-5.
   d. Постройте вертикальную бар-диаграмму, отображающую популярность жанров.
   e. Сохраните диаграмму в формате PDF.
   f. Выведите суммарные продажи для каждого жанра в консоль.
   g. Экспортируйте данные о топ-5 жанрах в CSV-файл.
3. Продажи по годам
   a. Загрузите датасет.
   b. Сгруппируйте данные по годам (Year), суммируя продажи по каждому году.
   c. Постройте линию тренда, отображающую изменение объема продаж по годам.
   d. Найдите годы с наибольшими и наименьшими продажами.
   e. Сохраните график в формате SVG.
   f. Выведите суммарные продажи для каждого года в консоль.
   g. Экспортируйте данные о продажах по годам в Excel-файл.
4. Региональные предпочтения
   a. Загрузите датасет.
   b. Постройте график, сравнивающий продажи игр в Северной Америке (NA_Sales), Европе (EU_Sales) и Японии (JP_Sales).
   c. Определите регион с наибольшими продажами и регион с наименьшими продажами.
   d. Найдите игры, которые были особенно популярны в конкретном регионе.
   e. Сохраните график в формате JPEG.
   f. Выведите суммарные продажи для каждого региона в консоль.
   g. Экспортируйте данные о региональных продажах в JSON-файл.
5. Самые продаваемые игры
   a. Загрузите датасет.
   b. Отсортируйте игры по общему объему мировых продаж (Global_Sales).
   c. Выведите топ-10 игр с указанием их названия, жанра и платформы.

    d. Постройте круговую диаграмму, показывающую долю каждой игры в общих продажах.
    e. Сохраните диаграмму в формате PNG.
    f. Выведите суммарные продажи для каждой игры в консоль.
    g. Экспортируйте данные о топ-10 играх в HTML-файл.
6. Влияние издателя на продажи
    a. Загрузите датасет.
    b. Сгруппируйте данные по издателям (Publisher), суммируя продажи по каждому издателю.
    c. Отсортируйте издателей по суммарным продажам и выведите топ-5.
    d. Постройте горизонтальную бар-диаграмму, отображающую продажи по издателям.
    e. Сохраните диаграмму в формате PDF.
    f. Выведите суммарные продажи для каждого издателя в консоль.
    g. Экспортируйте данные о топ-5 издателях в CSV-файл.
7. Жанры и платформы
    a. Загрузите датасет.
    b. Постройте тепловые карты, показывающие популярность жанров на разных платформах.
    c. Найдите платформу с наибольшим разнообразием жанров и платформу с наименьшим разнообразием.
    d. Сохраните тепловые карты в формате PNG.
    e. Выведите суммарные продажи для каждой комбинации жанра и платформы в консоль.
    f. Экспортируйте данные о жанрах и платформах в Excel-файл.
8. Корреляция между продажами и годом выпуска
    a. Загрузите датасет.
    b. Постройте диаграмму рассеяния, отображающую зависимость мировых продаж от года выпуска.
    c. Рассчитайте коэффициент корреляции Пирсона между этими двумя переменными.
    d. Сохраните диаграмму рассеяния в формате SVG.
    e. Выведите коэффициент корреляции в консоль.
    f. Экспортируйте данные о продажах и годах в CSV-файл.
9. Игры с наибольшими региональными различиями
    a. Загрузите датасет.
    b. Рассчитайте разность в продажах между Северной Америкой, Европой и Японией для каждой игры.
    c. Отсортируйте игры по величине разницы и выведите топ-10 игр с наибольшими региональными различиями.
    d. Постройте столбчатую диаграмму, отображающую региональные различия для топ-10 игр.
    e. Сохраните диаграмму в формате JPEG.
    f. Выведите разность в продажах для каждой игры в консоль.

g. Экспортируйте данные о региональных различиях в JSON-файл.

10. Популярные жанры по годам
    a. Загрузите датасет.
    b. Сгруппируйте данные по жанрам и годам, суммируя продажи по каждому жанру и году.
    c. Постройте график, отображающий изменение популярности жанров по годам.
    d. Найдите жанры, которые стали более популярными в последние годы.
    e. Сохраните график в формате PNG.
    f. Выведите суммарные продажи для каждого жанра и года в консоль.
    g. Экспортируйте данные о жанрах и годах в Excel-файл.
11. График зависимости продаж от года выпуска
    a. Загрузите датасет.
    b. Постройте график, отображающий зависимость мировых продаж от года выпуска.
    c. Добавьте заголовок и подписи осей.
    d. Сохраните график в формате PNG.
    e. Выведите коэффициенты уравнения линейной регрессии в консоль.
    f. Экспортируйте данные о продажах и годах в CSV-файл.
12. Диаграмма рассеяния для региональных продаж
    a. Загрузите датасет.
    b. Постройте диаграмму рассеяния, отображающую продажи в Северной Америке, Европе и Японии.
    c. Добавьте легенду и подписи осей.
    d. Сохраните диаграмму в формате SVG.
    e. Выведите координаты точек на графике в консоль.
    f. Экспортируйте данные о региональных продажах в JSON-файл.
13. Тепловая карта жанров и платформ
    a. Загрузите датасет.
    b. Постройте тепловую карту, отображающую популярность жанров на разных платформах.
    c. Добавьте цветовую шкалу и подписи осей.
    d. Сохраните тепловую карту в формате PNG.
    e. Выведите значения тепловой карты в консоль.
    f. Экспортируйте данные о жанрах и платформах в Excel-файл.
14. Фильтрация и сортировка данных
    a. Загрузите датасет.
    b. Отфильтруйте данные, оставив только игры, выпущенные после 2010 года.
    c. Сгруппируйте данные по жанрам, суммируя продажи по каждому жанру.
    d. Отсортируйте жанры по суммарным продажам и выведите топ-5.
    e. Постройте вертикальную бар-диаграмму, отображающую популярность жанров.
    f. Сохраните диаграмму в формате PDF.

g. Выведите суммарные продажи для каждого жанра в консоль.
h. Экспортируйте данные о топ-5 жанрах в CSV-файл.

15. Корреляционный анализ
    a. Загрузите датасет.
    b. Рассчитайте корреляционную матрицу между всеми количественными признаками (продажами, годом выпуска и т.д.).
    c. Постройте тепловые карты для визуализации корреляционной матрицы.
    d. Сохраните тепловые карты в формате PNG.
    e. Выведите корреляционные коэффициенты в консоль.
    f. Экспортируйте корреляционную матрицу в Excel-файл.

Описание набора данных Titanic

Структура набора данных:

1. **PassengerId:** Уникальный индекс/номер строки. Начинается с 1 (для первой строки) и увеличивается на 1 для каждой следующей. Рассматриваем его как индентификатор строки и, что логично, идентификатор пассажира (т.к. для каждого пассажира в датасете представлена только одна строка).

2. **Survived:** Признак, показывающий был ли спасен данный пассажир или нет. 1 означает, что удалось выжить, и 0 - не удалось спастись.

3. **Pclass:** Класс билета. 1 - означает Первый класс билета. 2 - означает Второй класс билета. 3 - означает Третий класс билета.

4. **Name:** Имя пассажира. Имя также может содержать титулы и обращения. "Mr" для мужчин. "Mrs" для женщин. "Miss" для девушек (тут имеется в виду что для тех, кто не замужем, так было принято, да и сейчас тоже, говорить в западном обществе). "Master" для юношей.

5. **Sex:** Пол пассажира. Либо мужчины (=Male) либо женщины (=Female).

6. **Age:** Возраст пассажира. "NaN" значения в этой колонке означают, что возраст данного пассажира отсутствует/неизвестен/или не был записан в датасет.

7. **SibSp:** Количество братьев/сестер или супругов, путешествующих с каждым пассажиром.

8. **Parch:** Количество родителей детей (Number of parents of children travelling with each passenger).

9. **Ticket:** Номер билета.

10. **Fare:** Сумма, которую заплатил пассажир за путешествие.

11. **Cabin:** Номер каюты пассажира. "NaN" значения в этой колонке указывает на то, что номер каюты данного пассажира не был записан.

12. **Embarked:** Порт отправления данного пассажира.

Описание набора данных Video Game Sales (vgsales.csv**)**

Video Game Sales — это популярный набор данных, часто используемый для анализа продаж видеоигр. Этот датасет содержит информацию о продажах игр на различных платформах (PlayStation, Xbox, Nintendo и др.), включая данные о названии игры, жанре, платформе, издателях, региональных продажах и общем объеме продаж.

Датасет обычно используется для выполнения различных аналитических задач, таких как:

- Исследование популярности платформ и жанров.
- Анализ региональных различий в предпочтениях игроков.
- Прогнозирование успеха игр на основе предыдущих данных.
- Исследование влияния критиков и пользователей на продажи.

Структура набора данных:

1. **Rank**: Ранг игры в списке продаж.
2. **Name**: Название видеоигры.
3. **Platform**: Платформа, на которой игра была выпущена (например, PlayStation, Xbox, Wii и т.д.).
4. **Year**: Год выпуска игры.
5. **Genre**: Жанр игры (например, Action, Adventure, RPG и т.д.).
6. **Publisher**: Издатель игры.
7. **NA_Sales**: Продажи игры в Северной Америке (в миллионах единиц).
8. **EU_Sales**: Продажи игры в Европе (в миллионах единиц).
9. **JP_Sales**: Продажи игры в Японии (в миллионах единиц).
10. **Other_Sales**: Продажи игры в остальных регионах мира (в миллионах единиц).
11. **Global_Sales**: Общие мировые продажи игры (сумма продаж по регионам).